

Speaker Notes

Slide 1 - Title

Start simple:

- > This presentation is about the RL work I did for the Furuta inverted
- > pendulum: what worked, what did not, what I learned, and how someone can
- > continue from the GitHub repository.

Do not overclaim. The honest headline is better:

- > A simulation-trained TD3 policy transferred far enough to attempt swing-up on
- > real hardware. The remaining problem is robust and quiet capture near upright.

Slide 2 - What Started The Project

Explain that the Furuta pendulum is a difficult nonlinear underactuated system:

- one motor at the rotary arm,
- pendulum is indirectly controlled,
- swing-up needs energy injection,
- balance needs fine stabilization.

The goal was not only to get one lucky video. The goal became a workflow:

- train,
- evaluate,
- compare model and hardware,
- document,
- hand over.

Slide 3 - System And Control Path

Emphasize the action interpretation because this caused many questions:

- > The RL agent outputs a normalized action. On the hardware-oriented path, that
- > action becomes a current command. It is not a torque command directly on the
- > real system.

Mention:

- the uC has its own current PI loop,
- the RL policy does not need to run at the 20 kHz firmware rate,
- 200 Hz and 500 Hz policy rates were tested.

Slide 4 - Why TD3

Say TD3 is a practical continuous-action baseline:

- DDPG was too fragile in our early tests,
- TD3 keeps a deterministic actor but improves training stability with twin critics and delayed policy updates,
- actor-only deployment is convenient.

Avoid going deep into math unless asked.

Slide 4b - DDPG vs TD3 vs SAC vs PPO

Use this slide if someone asks, "why TD3 and not SAC/PPO?"

Plain-language version:

- DDPG learns a deterministic continuous controller from replay-buffer data.

It is conceptually simple, but in this project it was fragile. It produced high-frequency torque and did not pass fixed local tests reliably.

- TD3 is basically a more robust DDPG family method. It still gives a deterministic continuous actor, which is convenient for deployment as a current-command policy. The two critics, delayed policy updates, and target smoothing directly address the DDPG failure mode.

- SAC is also off-policy and continuous, but the actor is stochastic during training and it optimizes reward plus entropy. That can make exploration better, especially for swing-up, but it changes more assumptions and usually uses a larger stochastic policy. It remains a good future option.

- PPO is different: it is on-policy. It collects trajectories with the current policy and then updates within a clipped/trust-region-like objective so the policy does not change too violently. PPO can handle discrete or continuous actions. In the Brian Douglas/QUBE hybrid example, PPO is used for discrete mode selection, not for motor current directly.

Good concise line:

- > DDPG failed first, TD3 was the closest controlled improvement, SAC was a
- > reasonable future alternative, and PPO fit the hybrid switching problem more
- > than our direct continuous-current controller.

Slide 5 - Observation And Action Design

This is one of the important lessons:

- > For swing-up, raw wrapped angles are awkward because the pendulum error jumps
- > at the wrap boundary. Using sine and cosine gives the policy a continuous
- > global angle representation.

Mention that 'previousAction' was added because:

- action-difference is in the reward,
- actuator dynamics make the previous command relevant,
- it helps the observation be closer to Markov.

Slide 6 - Early Attempts And Lessons

Frame early failures as useful:

- > The early DDPG and TD3 attempts were not wasted. They revealed that training
- > reward was not enough, raw local observations were insufficient for swing-up,
- > and in-episode learning could make Simulink training very expensive.

Mention learning frequency:

- DDPG initially learned very frequently,
- TD3 update cost became large when episodes survived longer,
- 'LearningFrequency = -1' moved learning to episode end.

Slide 7 - Key Shift: MathWorks-Style TD3

This is the first big success point.

Explain:

- no curriculum at first,
- fixed broad reset,
- fixed evaluation grid,
- episode-end learning,
- default 64-64 actor/critic style.

Important phrasing:

- > The MathWorks-style setup did not magically solve RL. It made the experiment
- > clean enough that TD3 could learn and that the evaluation became meaningful.

Slide 8 - Best Simple-Model Baselines

Use this slide to define the two important baseline agents.

200 Hz wide:

- useful first strong baseline,
- full 297-case evaluation,
- failures mainly arm-limit related.

500 Hz PI/current long:

- closer to hardware deployment,
- corrected evaluation after sample-time workspace bug,
- best hardware candidate so far.

Say:

- > This 500 Hz long policy is the one that became the main hardware-tested
- > policy.

Slide 8b - 500 Hz Long Training Config

Use this slide if someone asks what "500 Hz long" actually means.

The important distinction is that there are two time scales:

- the plant simulation still ran at '5e-5 s', so the model/PI loop side was effectively 20 kHz,
- the RL policy only chose a new action every '0.002 s', so the agent acted at 500 Hz.

For one training episode:

$$\text{EpisodeDuration} / \text{AgentSampleTime} = 5 \text{ s} / 0.002 \text{ s} = 2500 \text{ agent steps}$$

That means a full-length episode gives 2500 transitions to the replay buffer.

An episode can end earlier if the Simulink 'isDone' logic trips, for example because the arm or pendulum leaves the allowed training envelope. For hardware, we may tolerate small boundary violations more carefully, but during training 'isDone' matters because it shapes what the agent thinks is terminal failure.

Explain the learning settings plainly:

- 'LearningFrequency = -1' means MATLAB does not update the neural networks every few agent steps during the episode. It collects the episode, then runs learning at the end.
- 'MiniBatchSize = 1024' means each gradient update samples 1024 transitions from the replay buffer.
- 'NumEpoch = 10' means the end-of-episode learning pass can repeat for 10 epochs.
- 'MaxMiniBatchPerEpoch = 100' limits each epoch to at most 100 minibatches.

So the maximum number of minibatch updates over the full configured run is:

$$\begin{aligned} & \text{MaxEpisodes} * \text{NumEpoch} * \text{MaxMiniBatchPerEpoch} \\ &= 5000 * 10 * 100 \\ &= 5,000,000 \text{ minibatch updates} \end{aligned}$$

This is an upper bound. Actual work can be lower if the replay buffer is not ready early on, if episodes terminate quickly, or if MATLAB internally stops a learning pass because there is not enough useful data. It is also not the same as environment steps. The maximum environment interaction is:

$$5000 \text{ episodes} * 2500 \text{ steps} = 12,500,000 \text{ agent steps}$$

In TD3, the critic is updated more often than the actor. With 'PolicyUpdateFrequency = 2', the actor update is delayed and only happens every second critic-update step. This is one of the reasons TD3 is usually more stable than plain DDPG.

Training time:

- For the exact original '500Hz_long' run, the config and saved artifacts are preserved, but the docs do not currently contain a precise wall-clock diary with start/end timestamps.
- A closely related 500 Hz long actor1x64/critic2x64 comparison run took about '3 h 36 min' from initialization to final save.
- The later detailed-model 5000-episode runs were documented as "several hours" of active training, with timestamp-derived wall-clock estimates contaminated by idle/wait time.

So say this carefully:

- > The order of magnitude for a full 500 Hz long training run was several hours,
- > not minutes. The exact baseline runtime was not logged cleanly enough to quote
- > as a hard number.

Slide 9 - Hardware Transfer

This is the strongest practical result.

Say:

- > The policy did not balance perfectly, but it did enough on hardware that the
- > problem is clearly not just sign conventions or broken wiring.

Mention what was observed:

- attempted lift-up,
- sometimes hardware looked better than matching simulation,
- jitter near upright,
- scaling the current reduced jitter.

Slide 10 - Hardware Data Recorded

Explain why this matters:

- > This turned the hardware test from a one-off subjective observation into data
- > that can be replayed, averaged, compared, and used later.

Mention:

- 'scale0p4' had max commanded current around 1.5-1.6 A,
- 'scale1p0' reached about 3.9 A command and about 3.2 A measured current,
- all runs are saved with metadata and plots.

Slide 11 - Model vs Hardware Diagnostics

Talk about Test 1:

- > Before blaming the RL agent, I wanted a simple open-loop test where the input
- > is known. A 0.2 A command and then motor disable tells us a lot about friction
- > and free response.

Point:

- before disable, broad signs/dynamics matched,
- after disable, mismatch grew,
- this motivated the detailed model.

Slide 12 - Weto Meeting Inputs

Explain this as the turning point after the external feedback:

- > After the meeting, I tested the suggestions as controlled experiments rather
- > than changing everything at once.

List the categories:

- smaller network,
- reduced observation,
- detailed model,
- state-dependent reward,
- domain randomization later.

Then give the outcomes. This is a useful "we actually tested the advice" slide:

1. Smaller network:

- Actor 1x64 / critic 1x64 failed.
- It mostly failed by barely acting, so it was probably underfit or the critic was too weak.
- Actor 1x64 / critic 2x64 learned. So the small actor idea was viable, but the critic needed capacity.
- It was not the new hardware candidate because balance metrics were worse.

2. Reduced observation:

- Pure unsigned arc distance was elegant but removed direction.
- The run failed/interrupted and was still terminating after tens of steps.
- Lesson: one scalar per angle is attractive, but not if it makes left/right or positive/negative error ambiguous.

3. Detailed model:

- This was the most valuable path.
- It added current noise, friction, filters, encoder effects, current PI behavior, and dead-zone compensation.
- The first result was not better control, but it exposed the real next problem: the old reward can learn survival/swinging instead of capture.

4. Domain randomization:

- Initially attractive after hardware mismatch.
- Paused deliberately because the nominal detailed model still does not learn clean capture.

Transition:

- > Weto's input helped turn the project from "try more training" into controlled
- > experiments: network, observation, detailed model, then reward and robustness.

Slide 12b - Why Not The Brian Douglas Hybrid SAC/PPO Route?

This is worth explaining because it is a very reasonable question.

The Brian Douglas / MathWorks QUBE Servo2 reference uses a hybrid architecture:

- classical feedback controller handles upright balance,
- SAC learns swing-up behavior/reference generation,
- PPO chooses the mode, essentially switching between swing-up and the balance controller.

Why this is attractive:

- it uses classical control where classical control is strong,
- it gives RL a narrower job,
- it avoids asking one neural policy to solve swing-up, capture, and stable balance all at once.

Why we did not follow it directly:

1. Project objective.

The goal here was to explore whether a direct RL controller could learn the Furuta swing-up/balance task and transfer to hardware. If we used a classical controller for balance from the start, the final result would be less about direct RL control and more about RL-assisted switching.

2. Integration complexity.

The Brian Douglas example is built for Quanser QUBE/Raspberry Pi and its own Simulink architecture. Our setup uses the ZHAW Furuta model, SLDRT, UART, a microcontroller current PI loop, and different hardware signals. Porting the full hybrid architecture would be a project by itself.

3. Debuggability.

The hybrid design has at least three moving pieces: SAC swing-up, PPO mode selector, and classical balance controller. Early in our project, we were still debugging signs, reward, sample time, evaluation, and actuator interface. A direct TD3 baseline was easier to diagnose.

4. Controlled algorithm path.

We started with DDPG. TD3 was the smallest algorithmic step that addressed DDPG's known weakness while keeping the same deterministic continuous-action framing. SAC/PPO would have changed more at once.

Fair conclusion:

- > The hybrid SAC/PPO architecture is probably a strong future fallback or extension. In hindsight, it also supports one lesson from our work: swing-up
- > and balance may deserve different controllers, rewards, or modes.

Slide 13 - Smaller Network Experiments

Important nuance:

- > A smaller actor is feasible, but a too-small critic failed.

Say:

- actor 1x64 / critic 1x64 did not act enough,
- actor 1x64 / critic 2x64 learned,
- but it was not better for upright balance.

Lesson:

- > Critic capacity matters during training even if we want a small deployed
- > actor.

Slide 14 - Reduced Observation Experiments

Explain the topology tradeoff:

- > For a circular angle, one scalar cannot be globally continuous, directional,
- > and compact at the same time. We tried an unsigned arc-distance scalar, but it
- > lost too much directional information.

Main lesson:

- > Smaller observation is not automatically better. The policy needs the
- > information required to choose direction.

Slide 15 - Detailed Model Work

This slide is about engineering realism.

Say:

- > The simple model was useful for learning, but hardware has friction,
- > filtering, current-loop dynamics, quantization, and dead zones. The detailed
- > model tries to represent those before adding randomization.

Mention the principle:

- > Do not randomize a model that cannot solve the nominal task yet.

Slide 16 - Detailed Model Results

This is important and slightly subtle:

- > The detailed model runs often had zero failure rate, but that did not mean
- > success. The agent survived by swinging continuously.

Also mention the concrete failure mode from the last week:

- > One agent learned a strange sideways-survival behavior: it did not really
- > swing up, but it could hold or hover the pendulum around a sideways region and
- > jitter there. From the training reward's point of view this was better than
- > failing quickly, but from the control objective it was clearly wrong.

Use the numbers:

- 'MeanFinalTheta2MAE = 1.645 rad',
- 'MeanTheta2EndPeakToPeak = 6.235 rad', almost one full revolution,
- 'ActionDiffRMS' low, so the action was smooth,
- but smooth wrong behavior is still wrong.

This is a great lesson:

- > Safety survival is not the same as control objective success.

Slide 17 - Current Technical Bottleneck

Restate the new problem:

- > We are past the question of whether TD3 can move the pendulum. The question
- > now is capture: how to swing up, enter the upright region, damp velocities,
- > and stay there without jitter.

Mention likely fixes:

- capture reward,
- state-dependent penalties,
- current/action smoothing near upright,
- residual/teacher-policy fine-tuning.

Slide 18 - Do's

This slide should feel like a postmortem, not generic advice.

Start with:

- > These do's are not theoretical. Each one comes from something that bit us in
- > this project.

Specific points to mention:

1. Fixed evaluation was essential.

Training reward looked good multiple times while the controller was not actually good. The narrow MathWorks-style run had no failures but poor final theta2 accuracy. The detailed 1c run had zero failure but was still swinging around instead of balancing.

2. Look beyond failure rate.

Say:

- > A controller can be safe but bad. On the detailed model the agent stayed
- > inside limits, but the end-window pendulum peak-to-peak was about one full
- > revolution.

Mention metrics:

- final theta2 error,
- end-window theta2 oscillation,
- action-difference RMS,

- electrical energy,
- current saturation.

3. Match the experiment configuration.

We had a real bug where parallel 500 Hz evaluation workers used a default 200 Hz workspace config. That made old 500 Hz eval artifacts unreliable. The fix was carrying the saved run config into workers.

4. Change one factor at a time.

The useful experiments were controlled:

- small actor with same reward,
- critic restored after actor/critic 1x64 failed,
- arc observation tested separately,
- detailed model tested before domain randomization.

5. Smoke-test loaded agents.

A loaded TD3 agent can bring old exploration, warm-start, and option values. Before fine-tuning, run a zero-exploration smoke test to check whether the policy itself is stable.

6. Log hardware like data, not like a demo.

The 2026-07-07 hardware runs became useful because they were saved with metadata, summary CSVs, quicklook plots, and aligned averages.

Slide 19 - Don'ts

Make this practical and project-specific:

- > The most dangerous mistake is to trust training reward or failure rate alone.

Use these examples:

1. Do not present survival as balance.

The detailed 1c 7000-episode run is the clean example:

```
FailureRate = 0
MeanFinalTheta2MAE = 1.645 rad
MeanTheta2EndPeakToPeak = 6.235 rad
```

It learned a smooth swinging strategy, not capture.

2. Do not trust old 500 Hz evaluations blindly.

The parallel-worker config bug means old sample-time-sensitive 500 Hz comparisons must be checked. Prefer corrected 'full_fixed_workspace_*. files.

3. Do not shrink the critic too aggressively.

Actor 1x64 / critic 1x64 failed and barely acted. Actor 1x64 / critic 2x64 learned. The critic is part of training, even if we only deploy the actor.

4. Do not remove direction information just to reduce observation dimension.

The unsigned arc-distance observation was elegant, but made '+angle' and '-angle' identical. That hurt especially for theta1 arm centering.

5. Do not add domain randomization too early.

We wanted to randomize after hardware mismatch, but the detailed nominal model still did not learn capture. Randomization should make a working nominal controller robust; it should not hide a base-task failure.

6. Do not compare hardware and simulation with different action scaling.

Hardware scale '0.4' was a different effective controller from raw training scale '1.0'. That is fine as a safety deployment test, but it must be named explicitly.

Slide 20 - Handover: Repository Structure

Show the repository.

Point to:

- 'docs/' first,
- 'scripts/make*Config.m' for training definitions,
- 'scripts/train*.m' for launchers,
- 'scripts/analysis/' for hardware/simulation save and compare,
- 'results/TD3/' for runs,
- 'results/model_vs_hardware/' for hardware data.

Say:

- > The most important thing for continuation is to start from the documented
- > baseline, not from a random latest file.

Slide 21 - Demo Plan

Suggested demo if time is short:

1. Open the hardware swing-up average plot.
2. Open the model-vs-hardware comparison plot.
3. Show the SLDRT/current command Simulink path.
4. Show the result folder structure.

If hardware is connected:

- only run if safety setup is ready,
- use reduced current scaling,
- log the run.

Slide 22 - Future Work

State the recommended next step:

- > Do not add broad domain randomization yet. First make the nominal detailed
- > model capture and balance.

Then:

- capture-tune latest 1c detailed agent,
- add upright-region shaping,
- use policy-preserving fine-tuning,
- add actuator state if needed,
- then domain randomization.

Slide 22b - Sim-To-Real Research Directions

Connect this back to the current state:

- > We now have a policy that works on the simpler model and transfers partly to
- > hardware, but detailed-model fine-tuning can damage it. That makes this a
- > sim-to-real adaptation problem, not just a "train longer" problem.

Summarize the papers/ideas:

1. SimOpt / adaptive domain randomization.

Instead of manually guessing friction/current/dead-zone randomization ranges forever, use real rollouts to adapt the simulator distribution. This matches the idea of collecting hardware logs, comparing them to simulation, and tightening or shifting the randomization.

2. Residual reinforcement learning.

Keep a useful base controller and learn a correction. This is attractive because the '500Hz_long' policy already has useful swing-up behavior. A

residual policy or residual action near upright may be safer than letting TD3 overwrite everything.

3. EPOpt / robust model ensembles.

Train across a family of model variants and focus learning on difficult cases. This becomes relevant once the nominal detailed model can balance.

4. Multi-fidelity RL.

Structure the learning path from cheap/simple simulation to detailed simulation and then hardware. This describes our practical workflow well: simple 1b model -> detailed 1c/1d model -> hardware data.

5. Policy distillation / learning without forgetting.

Our fine-tuning collapse suggests we need to preserve old behavior while adapting. A teacher-policy action-deviation penalty is the practical version of this idea for our MATLAB workflow.

Final research takeaway:

- > The next step should preserve the working simple-model policy, adapt gently
- > on the detailed model, and use hardware data to choose mismatch ranges. Broad
- > randomization comes after nominal capture works.

Slide 23 - Final Takeaway

End with a balanced message:

- > The project produced a working RL training/evaluation/hardware logging
- > workflow and a policy that reached hardware swing-up attempts. The remaining
- > work is a well-defined sim-to-real robustness and capture problem.

This is a good place to thank Ruprecht, Michi, and Weto for guidance.